CS2310 Take-home 6: Memory, branches, advanced instructions, integration

Group 38 | Roll number: **CS25B035** | Logisim-Evolution 4.1.0 | G38_th6_tara.circ, G38_th6_microcode_gen.py, G38_th6_control_rom.hex, G38_th6_tests/

Summary. The TH5 processor is extended with the eleven remaining graded instructions: LDW, STW, LDB, STB, JMP, BZ, BN, CALL, RET, PUSH and POP. Only two pieces of hardware were added: a **BYTE_MODE** byte-lane unit inside ram_word (byte read with zero-extension, and a write enable per byte bank) and a **register-file write-index multiplexer** that can force the destination to R6 or R7. Everything else is microcode. All 26 graded instructions pass their own tests, the documented boundary cases and the regression suite; all six integration programs run to HLT with registers, PC and all 2048 bytes of memory identical to TARA Studio (27/27 programs). The generator passes 7/7 consistency checks. Memory, branch and advanced pools claimed: 57/57.

1. Interface table (subcircuits changed in TH6; ports exactly as labelled in the file)

Subcircuit	Inputs	Outputs	Change in TH6
ram_word	addr[16], din[16], mem_read, mem_write, clk, **BYTE_MODE**	dout[16]	byte read path (A0 mux + zero-extend), per-bank write enables, high-bank data mux
tara_datapath	CLK, RST, PC_OUT, IR_OUT, MDR_OUT, IMM_OUT, RF_OUT, ALU_OUT, RAM_OUT_LD, MAR_LOAD, MDR_LOAD, IR_LOAD, ACC_load, RF_WRITE, PC_EN, PC_LOAD, mem_read, mem_write, ALU_select[5], ALU_A_SEL, ALU_B_SEL, status_we, IMM_SEL[3], RA_SEL[2], **RF_IDX_SEL[2]**, **BYTE_MODE**	opcode[5], Z, N, PC[16], IR[16]	write-index mux (RD, 6, 7, RD) in front of regfile rd; BYTE_MODE to ram_word
tara_cpu (main)	RST; Clock CLK	-	new 23-end control-word splitter; IR_OUT = constant 0; ACC_load = ALU_OUT
control_unit	CLK, RST, Z, N, opcode[5]	cw[32], T_STATE[4], HALTED, HALT	unchanged; ROM reloaded with TH6 image
imm_unit	imm8[8], off5[5], rel8[8], rel11[11], QA[16], IMM_SEL[3]	IMM[16]	unchanged; its sext(off5), rel8x2, rel11x2 and const 2 forms are now used

All other subcircuits (ALU, tara_decoder, regfile8x16, status_reg, PC_MAR_MDR_IR, register16, adders, shifters, comparators) are unchanged from TH5; their ports are listed in the TH5 report.

2. Control word, TH6 layout (32 bits, all used)

Bits	Field	Meaning	Bits	Field	Meaning
0	PC_OUT	PC → bus	12	mem_read	memory output enable
1	MDR_OUT	MDR → bus	13	mem_write	M[MAR] ← MDR (byte or word)
2	IMM_OUT	imm_unit → bus	14-18	ALU_select	ALU opcode
3	RF_OUT	port A (qa) → bus	19	ALU_A_SEL	ALU A: 0 qa, 1 PC
4	ALU_OUT	ALU → bus (also loads ACC)	20	ALU_B_SEL	ALU B: 0 qb, 1 IMM
5	RAM_OUT_LD	memory → bus	21	status_we	latch Z N C V
6	MAR_LOAD	MAR ← bus	22-24	IMM_SEL	0 zext8 1 sext8 2 sext5 3 rel8x2 4 rel11x2 5 const2 6 LIH 7 zero
7	MDR_LOAD	MDR ← bus	25-26	RA_SEL	port A: 0 IR[7:5], 1 IR[10:8], 2 R6, 3 R7
8	IR_LOAD	IR ← bus	27-28	RF_IDX_SEL	write index: 0 IR[10:8], 1 R6, 2 R7
9	RF_WRITE	R[index] ← bus	29	STEP_RESET	next edge: T-state ← 0
10	PC_EN	PC update (PC+2 if PC_LOAD=0)	30	HALT	stop, cw forced to 0
11	PC_LOAD	with PC_EN: PC ← bus	31	BYTE_MODE	ram_word byte access, lane = addr[0]

3. Design decisions

Decision	Justification
Byte access with per-bank write enables (no read-modify-write).	ram_word already holds two 8-bit RAMs (bottom = high byte, top = low byte). In byte mode only the bank selected by A0 = addr[0] is written: top Store = mem_write AND (NOT BYTE_MODE OR A0), bottom Store = mem_write AND (NOT BYTE_MODE OR NOT A0), built with negated OR-gate inputs. The bottom bank's data input takes din[7:0] in byte mode, so the low byte of Rs reaches either lane. The neighbouring byte is never rewritten, and STB needs 3 execute states, not 5.
LDB zero-extends inside ram_word.	In byte mode dout = zext(A0 ? low bank : high bank); otherwise dout = {high, low}. LDB therefore uses the same MDR path as LDW and only differs by BYTE_MODE at T4.
Big-endian, word accesses aligned.	M[even] is the high byte, as in TARA Studio. Word accesses use addr[15:1] and ignore addr[0]; all test programs access words at even addresses.
Effective address on the ALU.	EA = Rb + sext(off5): port A reads IR[7:5], ALU B = imm_unit form 2, result goes over the bus into MAR (T3). off5 is sign-extended, so -16 ... +15 (tested with -16, -1, +14).
PC-relative reference point is the incremented PC.	Fetch already leaves PC = address + 2, so JMP/BZ/BN/CALL compute PC + sext(rel) x 2 with ALU_A_SEL = 1 (PC) and IMM_SEL = 3 or 4 (the x2 is a zero wired below the field). This matches the assembler and TARA Studio.
BZ / BN in two states via status_reg.	T3: Rt + 0 through the ALU (port A reads IR[10:8], B = constant 0) with status_we, so Z and N describe Rt. Rt + 0 is used instead of Rt OR Rt because port B is wired to IR[4:2]. T4 is addressed by the latched Z (BZ) or N (BN): taken = PC ← PC + rel8x2, not taken = STEP_RESET only. The words differ only in that bit (generator check 5). BZ never depends on an earlier instruction (t14).
CALL writes the link first.	T3: R6 ← PC (PC_OUT, RF_WRITE, RF_IDX_SEL = 1); T4: PC ← PC + rel11x2. RET: port A forced to R6 (RA_SEL = 2), RF_OUT, PC_EN + PC_LOAD.
Stack: PUSH decrements then stores; POP loads then increments.	PUSH: R7 ← R7 - 2 (ALU SUB, const 2, RF_IDX_SEL = 2), MAR ← R7, MDR ← Rs, M[MAR] ← MDR. POP: MAR ← R7, MDR ← M, Rd ← MDR, R7 ← R7 + 2. PUSH R7 stores the decremented R7; POP R7 gives M[R7] + 2. Both equal TARA Studio (t17).
Control word re-laid out to fit 32 bits.	TH6 needs RF_IDX_SEL (2) and BYTE_MODE (1). IR_OUT was never used by any instruction and is tied to 0; ACC_load is tied to ALU_OUT (ACC is observational). STEP_RESET and HALT keep bits 29/30, so control_unit needed no change, only a new ROM image.

Decision	Justification
Unimplemented opcodes HALT.	MUL (12) and opcodes 27-31 get HALT at T3 and change no state (check 6, test t21).
Memory size.	ram_word addresses 32 K words; TARA has 2 KB. Programs use 0x000-0x7FF, so behaviour is identical; addresses above 0x7FF are not wrapped (documented limitation, never used by the tests).

4. TH6 microprograms (T0-T2 fetch: 0x00000041, 0x000014A0, 0x00000102)

Instr.	T3	T4	T5	T6
LDW	MAR ← Rb+sext(off5) 0x00924050	MDR ← M[MAR] 0x000010A0	Rd ← MDR, step-reset 0x20000202	
STW	MAR ← EA	MDR ← Rs (RA_SEL=1) 0x02000088	mem_write, step-reset 0x20002000	
LDB	MAR ← EA	MDR ← zext byte (BYTE_MODE) 0x800010A0	Rd ← MDR	
STB	MAR ← EA	MDR ← Rs	mem_write + BYTE_MODE 0xA0002000	
JMP	PC ← PC+rel11x2 0x211A4C10			
BZ / BN	Rt+0, status_we 0x03F24000	Z / N = 1: PC ← PC+rel8x2 0x20DA4C10; else 0x20000000		
CALL	R6 ← PC 0x08000201	PC ← PC+rel11x2 0x211A4C10		
RET	PC ← R6 0x24000C08			
PUSH	R7 ← R7-2 0x17528210	MAR ← R7 0x06000048	MDR ← Rs 0x02000088	M ← MDR 0x20002000
POP	MAR ← R7 0x06000048	MDR ← M 0x000010A0	Rd ← MDR 0x00000202	R7 ← R7+2 0x37524210

Generator log (G38_th6_microcode_log.txt):

```
TARA microcode generator, TH6 — 26 instructions: NOP HLT MOV LIL LIH ADD SUB ADDI AND OR XOR NOT SHL SHR SLT LDW STW LDB ST
ROM 2048 x 32 bits

[PASS] 1. no control word selects more than one bus source
[PASS] 2. every instruction eventually asserts STEP_RESET or HALT
[PASS] 3. mem_read and mem_write are never asserted together
[PASS] 4. write enables are never asserted without valid source data
[PASS] 5. conditional-branch words differ only in the relevant condition signal
[PASS] 6. no undefined opcode can modify architectural state (they HALT)
[PASS] 7. T0-T2 hold the common fetch sequence for every opcode

7/7 checks passed
```

5. Test evidence

Every program is assembled with the TARA Studio assembler, written into ram_word's two byte banks, and run on our circuit with Logisim's own simulator until HALTED = 1; the same program runs on TARA Studio. PASS means R0-R7, PC and all 2048 memory bytes are identical. Per-instruction traces of the six integration programs, recorded on our circuit, are in G38_th6_tests/traces; TARA Studio logs of every program are in reference_logs; screenshots of both for every integration program are in screenshots.

Program	Tests	R0 ... R7 after HLT (ours = TARA Studio)	PC	Result
t11	LDW/STW, off5 -16 and +14	0000 0410 1234 1234 0410 1234 0000 0410	001C	PASS
t12	LDB/STB, STB at odd and even address	00AB 0400 1234 0012 0034 99EE 12EE EE00	0022	PASS
t13	JMP forward and back	0000 0001 0002 0003 0000 0000 0000 0000	0012	PASS
t14	BZ taken / not taken, BZ on FFFF	0000 0000 FFFF 0000 0000 0004 0000 0000	0024	PASS
t15	BN on positive, zero, 8000	FFFF 0005 8000 0000 0003 0000 0000 0000	001E	PASS
t16	CALL/RET, nested with PUSH R6	0000 0052 0000 0000 0000 0014 000C 05F0	000E	PASS
t17	PUSH/POP, PUSH R7, POP R7	05F0 0011 AA22 AA22 0011 05EE 05EE 0013	0020	PASS
t18	POP at top of memory (07FE)	0000 0077 0077 0800 0000 0000 0000 0800	0012	PASS
t19	regression mix of all classes	0406 0000 0015 002A 002A FFC0 002A 05E6	002C	PASS
p1	register addition	0000 1234 0FED 2221 0000 0000 0000 0000	000C	PASS
p2	memory subtraction (M[404]=06A4)	0400 012C 07D0 012C 06A4 0000 0000 0000	001A	PASS
p3	parity of B6D5	0001 0000 0000 0001 0000 B6D5 0000 0000	001A	PASS
p4	123 x 45 by repeated addition	0000 007B 0000 159F 0000 0000 0000 0000	0010	PASS
p5	sum of 16-bit array = 1194	0400 0410 0000 1194 0000 007D 0000 0000	0030	PASS
p6	max of signed array = 05DC (CALL, PUSH, POP)	0400 040E 0000 05DC 0384 0000 0410 05F0	004C	PASS

Also passing on the same file: t01-t10, t20 (all 15 core instructions) and t21 (MUL halts): 27/27.

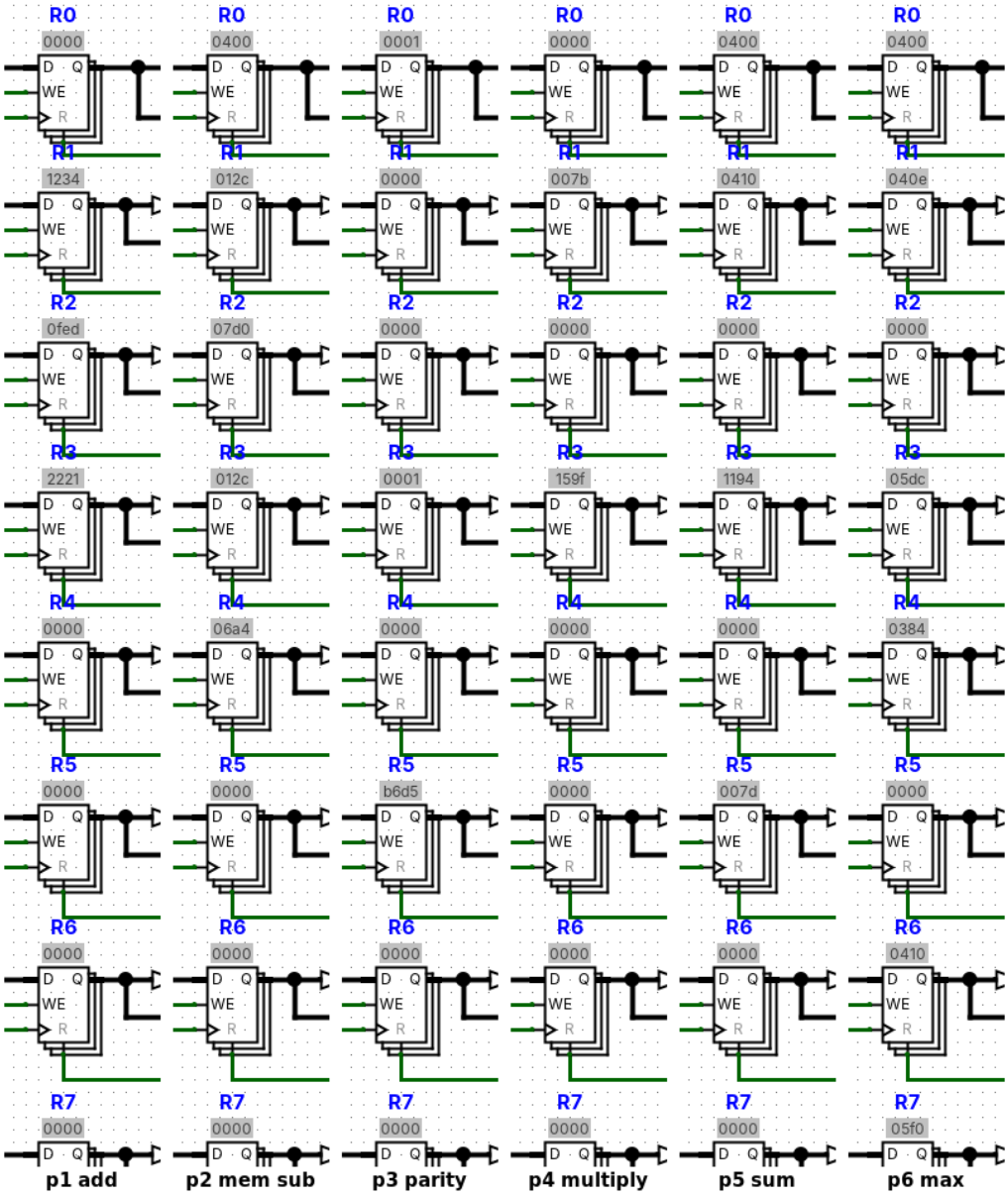


Figure 1. Register file of OUR circuit in Logisim after HLT for the six integration programs (R0 top to R7 bottom). p4: R3 = 159f = 123 x 45; p6: R3 = 05dc = 1500, R7 back at 05f0 after the CALL/PUSH/POP loop.

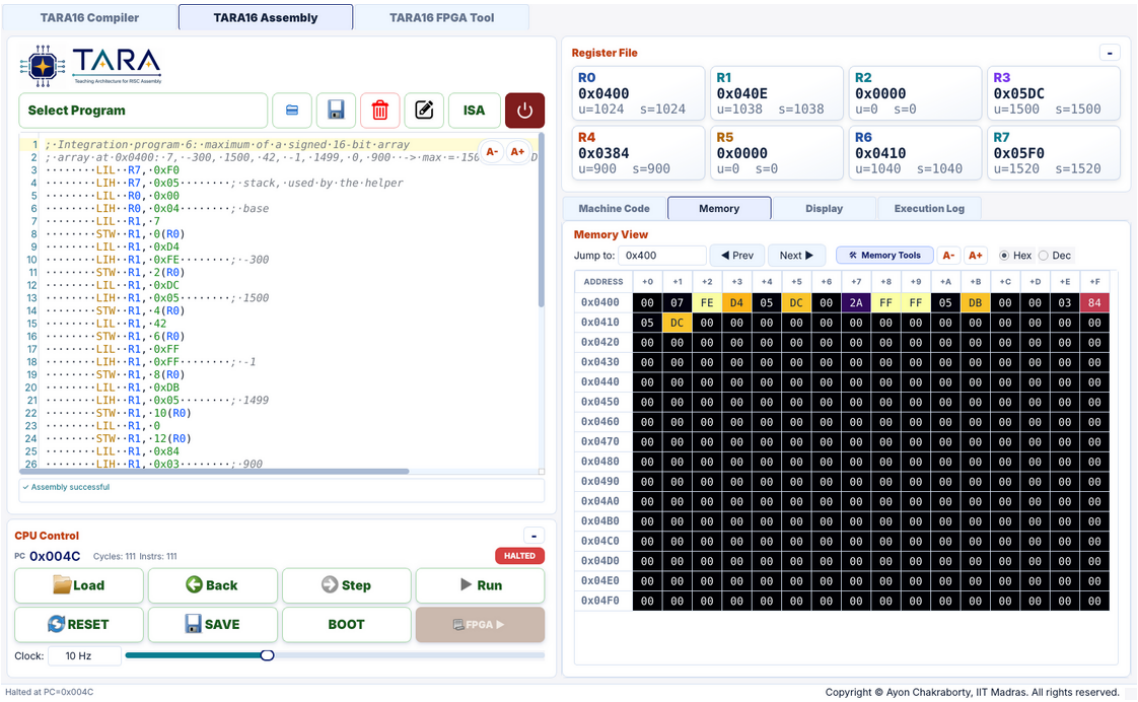


Figure 2. TARA Studio after p6 (HALTED, PC = 0x004C): same registers as Figure 1, memory at 0x400 shows the array and the result 05DC at 0x410. Screenshots for p1-p6 are in G38_th6_tests/screenshots.

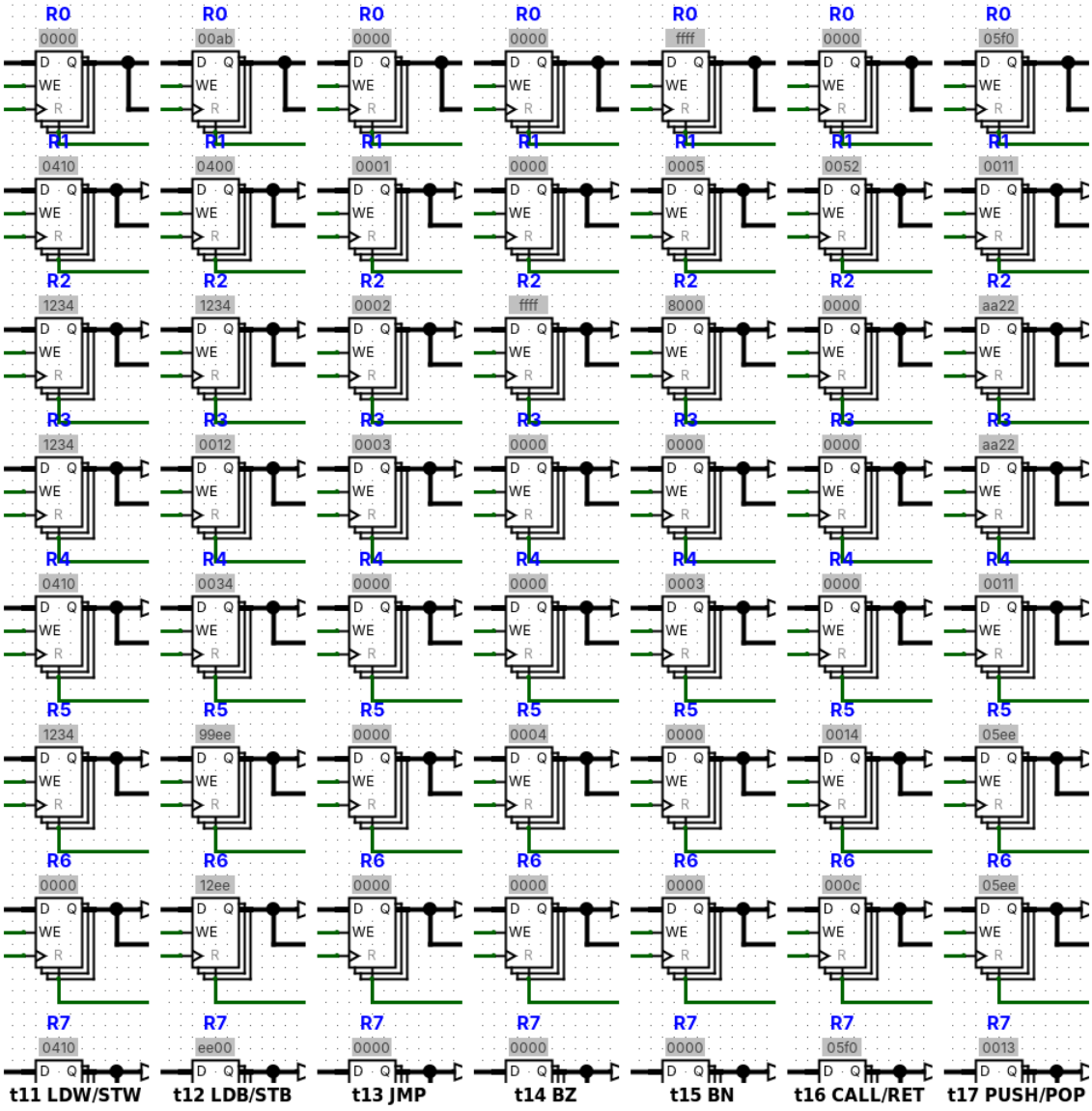


Figure 3. Our register file after the TH6 instruction tests. t12: R3 = 0012 / R4 = 0034 (LDB high and low byte), R6 = 12ee (STB at odd address kept the 12); t16: R6 = 000c link; t17: R7 = 0013 (POP R7).

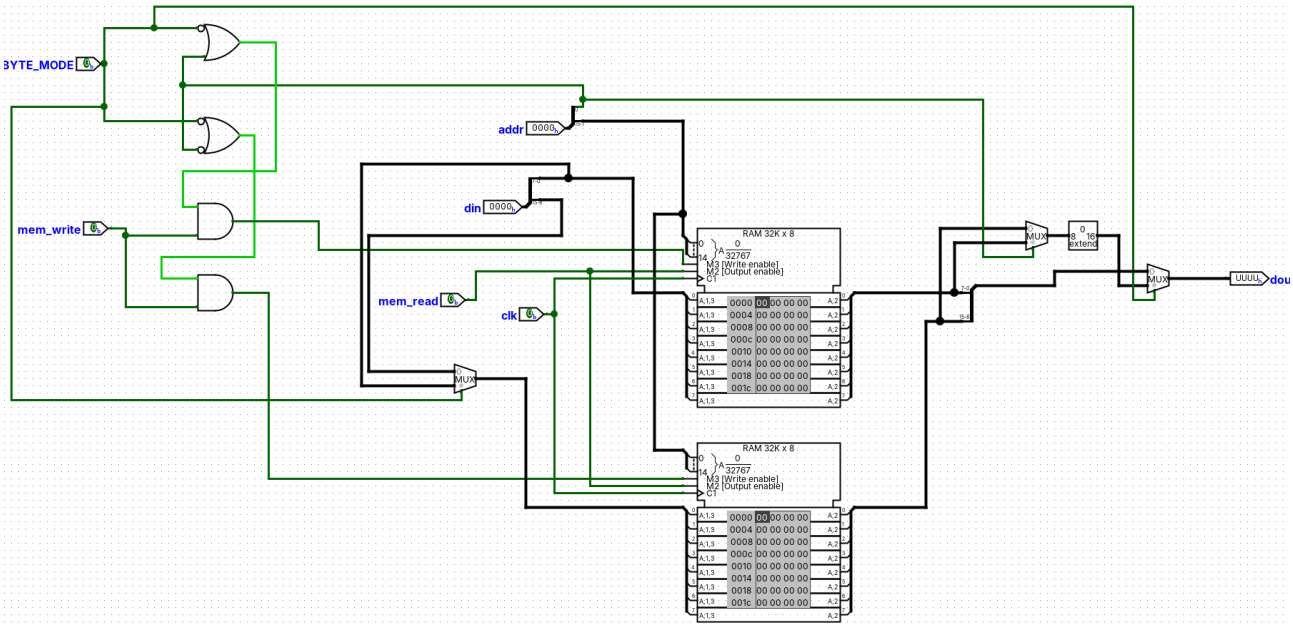


Figure 4. ram_word with the byte lanes: OR gates with negated inputs and AND gates give the two Store enables, the left mux feeds din[7:0] to the high bank in byte mode, the A0 mux + zero extender + right mux form dout.

6. Faults encountered and how they were found

Fault	Symptom, diagnosis and fix
Write-index mux select left floating.	After adding the RF write-index mux, a netlist check of tara_datapath showed its 2-bit select connected to nothing: there was no RF_IDX_SEL pin. Every register write would have gone to an undefined index. Added the RF_IDX_SEL input pin and connected it to splitter end 20 (bits 27-28) in tara_cpu.
Control word did not fit in 32 bits.	The TH5 word used 31 bits; RF_IDX_SEL and BYTE_MODE needed three more. Found while planning the layout. Solved by removing IR_OUT (no microprogram ever drives IR onto the bus; tied to 0) and deriving ACC_load from ALU_OUT, keeping STEP_RESET/HALT at bits 29/30 so control_unit stayed unchanged.
TH5 ROM image still loaded after the layout change.	The rewired tara_cpu was checked by comparing the ROM contents in the file with the generator output: the first words were still 0x81, 0x5140, 0x204 (TH5 fetch). With the new layout those bits mean other signals, so even fetch would fail. Reloading G38_th6_control_rom.hex gave 0x41, 0x14A0, 0x102 and an exact match over all 2048 words.
Main circuit set to the empty 'main'.	The saved file opened on an empty canvas because the main circuit was the old empty 'main', so a grader starting the simulation would see nothing. Set tara_cpu as main circuit.
Byte-lane test appeared to fail.	The first stand-alone test of ram_word returned 0000 for every read. Running the same test on the TH5 ram_word gave the same zeros, which located the fault in the test itself: its first write fell on the initial clock edge and was lost. With an idle first cycle the TH5 RAM passed the word cases (and failed the byte ones, as expected) and the TH6 RAM passed all seven cases, including STB at odd and even addresses with the neighbour byte preserved.